Functional Requirements & User Workflows

Document 04 of 07

Purpose
Define the user-facing behavior, inputs, outputs, validation, edge cases, cross-feature rules, and testable acceptance criteria for the current bulking assistant and its proposed next capabilities.

Product	Personal bulking assistant
Evidence snapshot	Repository default branch **rd2080/Bulk_App_Personalised**, branch **main**, reviewed 21 October 2026; local project mirror also reviewed.
Status legend	**Observed** = present in current GitHub docs/source evidence; **Planned** = requested capability not verified as implemented; **Undecided** = behavior needs product decision.
Important source note	Documents 02 and 03 were not available as readable files in this project mirror. This document uses accessible repository docs and README as context and records the limitation rather than assuming their contents.

How to use this document

Sections 2-5 describe observed behavior; sections 6-10 specify planned capabilities and acceptance criteria where implementation is absent or unclear. The requirements are product-level and testable. “Must” marks a proposed requirement for future delivery, not a claim that the application already satisfies it.

1. Scope and evidence

This is a functional handover, not a clinical nutrition protocol. The app currently uses Streamlit pages and Neon PostgreSQL persistence. The product goal is to help one user maintain profile and food context, organize planned meals, track daily signals, and eventually compare plans with actual intake.

Evidence hierarchy

- Current GitHub default branch README and feature docs establish the documented implementation snapshot.
- The local mirror contains tests and code for profile, check-in, recipe library, meal plan, request-builder foundation, voice transcription, and related repositories.
- The local mirror README and daily-meal-plan/voice docs are older than GitHub main: they describe the Phase 3 command path as not connected, while current GitHub docs describe deterministic commands as implemented. Treat command behavior as documented on main but not verified against the stale local checkout.
- Repository unit tests exist for profile, check-in, recipes, meal plan and request-builder foundation. This handover did not run the test suite; no test result is claimed. The PDF itself is verified separately.

Status at a glance

Capability	Evidence status	What is established
Profile	Observed	Single current profile stored in Neon; ten documented fields.
Daily check-in	Observed	Dated weight, sleep, soreness and notes; editable/reviewable by date.
Inventory + recipes + CSV	Observed	Food availability and recipe data; paired CSV import, templates, validation and transaction flow documented.
Dated meal planning	Observed	Plans per date; manual create/edit/remove/review, persisted.
Voice + deterministic commands	Documented on GitHub main; stale local mirror	Browser dictation and reviewed text; labeled initial plan and explicit add/replace/remove commands; ambiguity prompts; no AI call.
Actual meal logging	Missing	No actual consumption workflow verified.
Reminders	Missing / unspecified	No reminder scheduling or delivery verified.
Charts and analysis	Missing	No analytics/reporting workflow verified.
AI suggestions	Missing	Request-builder foundation only; no provider invocation or meal suggestion generation verified.

2. Shared interaction and data rules

- Use explicit dates for check-ins, meal plans, and future actual logs. Display the active date prominently before saving or applying a command.
- Persisted data is authoritative. Do not imply a save succeeded until persistence returns successfully; retain entered values and explain recoverable failures.
- Validate input before database writes. On invalid input, identify the field and reason; do not partially write related records.
- Separate planned meals from actual consumed meals. Editing a plan must never silently create, overwrite, or delete actual logs.
- Store quantities and units explicitly where applicable. Avoid deriving nutrient values from ingredient composition unless a verified calculation model exists.
- Use explicit confirmation before destructive bulk replacement or deletion; preview batch CSV changes before import.

3. Observed workflows

3.1 Profile

Input: age, height (cm), target weight (kg), diet type, diet philosophy, calorie strategy, cuisine preference, regional context, cooking complexity, and meals per day. **Output:** one current profile loaded from and saved to Neon. Saving validates the exact supported field set; numeric bounds and required text are checked in application and database layers. The AI request builder reads the current profile each call and creates structured request context, but does not invoke an AI provider or persist an AI copy.

3.2 Morning check-in

Input: check-in date, weight (kg), sleep hours, soreness level (1-10), optional daily notes. **Output:** saved/revised check-in for that date; weight is also written to the weight history. Date-specific review and update are documented. Invalid weight, sleep, or soreness values are rejected before database access. Exact accepted numeric ranges beyond the observed examples should follow the repository validators and be confirmed against product policy before expansion.

3.3 Food inventory

The inventory page adds/edits foods, manages availability categories, and shows available or unavailable items. The specific canonical food fields and category vocabulary are not fully established by the handover evidence available here; preserve existing schema and interface semantics. Recipes refer to existing inventory foods.

3.4 Recipe library and CSV

Recipes capture type (meal/snack/shake), preference (favorite/like/neutral/dislike/never), complexity (easy/medium/hard), prepared total weight, recipe-level calories/protein/carbohydrate/fat, and ingredient composition. Recipe names are unique; each food appears once per recipe; quantities and total weight are positive; nutrients are nonnegative. Ingredients reference inventory foods and do not store nutrition.

- Import requires both recipes.csv and recipe_ingredients.csv with exact ordered headers. Provide downloadable templates, validate and preview, then require a separate import confirmation.
- Food and recipe references are matched case-insensitively during validation; existing recipes are upserted by exact recipe name. The uploaded ingredient definition replaces that recipe’s ingredient list; a recipe with no ingredient rows becomes ingredient-free.
- Validation/import is documented as one transaction; on database failure, the full upload rolls back. Nutrition scaling is documented as recipe nutrient × serving weight ÷ total recipe weight.

3.5 Meal plan and command entry

The manual Daily Meal Plan holds one plan per date with any number of meals. Each row includes meal category, required name, optional details, and display position. Categories documented: breakfast, morning snack, lunch, afternoon snack, dinner, other. Users can add, edit, remove and reload meals for the selected date.

Current GitHub docs additionally describe Voice Input & Meal Commands: browser dictation using Web Speech API, editable transcript/text fallback, selected plan date, and deterministic Python parsing (no Gemini/OpenAI call). Commands include labeled multi-meal initial plan, “Change my lunch to …”, “Add … to breakfast”, and “Remove dinner.” Ambiguous generic snack, missing/repeated labels, or multiple matching existing entries ask for clarification. Meal contents remain text; this does not parse ingredients or nutrition.

4. Inputs and outputs by capability

Area	User inputs	Expected output
Profile	Ten profile fields; save intent	Current profile summary and persisted profile; field-level validation.
Check-in	Date, weight, sleep, soreness, notes	Saved date record; weight history updated; dated list/detail.
Inventory	Food attributes and availability; edit intent	Persisted food and availability-filtered listing.
Recipe	Recipe attributes, nutrition, weight, food ingredients	Recipe detail/list; scaled recipe nutrition when serving weight is requested.
CSV	Two files, exact headers and row data; explicit confirm	Validation findings and preview, then all-or-nothing import result.
Meal plan	Date, type, name, optional details, edits/deletes	Date-scoped ordered plan; persisted changes.
Voice/command	Spoken or typed text, reviewed transcript, selected date, apply	Transcript review; applied plan change or clarification/error; saved plan shown.
Future actual log	Date/time, food/recipe/free-text item, amount, unit (proposal)	Immutable or auditable consumed entry, editable with history (proposal).
Future analysis	Date range and selected metrics (proposal)	Charts/summaries with data coverage and source labeling (proposal).
Future reminders/AI	User-selected preferences and explicit request context (proposal)	Reminder delivery status or labeled AI suggestion with rationale (proposal).

5. Planned or missing functionality

The following capabilities were explicitly requested for this requirements handover but were not verified as implemented in the current repository evidence. Each item is a proposed product requirement to confirm in roadmap planning.

5.1 Actual meal logging

- Provide a separate “Log what I ate” workflow, distinct from plan editing. Capture consumed date/time, item (recipe, inventory food, or free text), amount, unit, optional note, and source (manual or command-assisted).
- Allow logging meals that were never planned and logging partial servings or multiple items. Link to a planned meal only when the user selects a match; never infer consumption from a plan.
- Allow correction/deletion with visible audit semantics to be decided. Show saved confirmation and daily actual log.

5.2 Reminders

- Allow opt-in reminders by event (check-in, meal plan review, meal logging), local time and selected weekdays; make timezone explicit and editable.
- Support pause, resume, and disable; show next scheduled reminder and last delivery outcome when available. Do not promise delivery until a delivery channel and scheduler are selected.
- Undecided: channel (in-app, email, push), quiet hours, retries, and whether reminders run in hosted or client environment.

5.3 Charts and analysis

- Offer date-range views for weight and check-in signals, and planned-versus-actual meal adherence only after actual logging exists.

- Display units, date range, missing-day gaps, and data counts. Do not interpolate missing check-ins as if observed.
- Nutrition totals/charts require verified nutrition data for logged portions. Clearly label estimates and items with unknown nutrition.

5.4 AI suggestions

- Keep deterministic commands separate from AI suggestions. Suggestions are advisory and require user review before saving to a plan or log.
- Use current profile plus explicitly approved context (availability, recipe preferences, recent check-ins, plan and actual log) and state which context was used. Do not claim context is available unless fetched successfully.
- Show generated timestamp, explanation, and any assumptions; provide regenerate, edit, accept and dismiss actions. Fail safely when no profile/context or provider is available.
- Undecided: provider/model, data retention, consent/privacy notice, nutrition source, constraints and safety/product wording.

6. Validation and edge-case requirements

Scenario	Required behavior
Blank/invalid required field	Block write; point to field; retain draft input.
Invalid date or future date	Validate consistently; future dates require an explicit product rule, not silent rejection.
Duplicate profile save	Update the single profile; do not create a second profile.
Repeated check-in for a date	Update that date’s record and corresponding weight entry without duplicating the date.
CSV header/order mismatch	Reject before writes; identify file and expected columns.
Unknown food/recipe reference or duplicate rows	Reject with row-level findings; no partial import.
CSV update without ingredient rows	Warn in preview that composition will be replaced with empty; require explicit confirmation.
Command ambiguity / unsupported phrasing	Ask a targeted clarification; leave plan unchanged until resolved.
Voice unavailable / permission denied / empty transcript	Keep manual text entry available; do not apply empty input.
No existing plan for selected date	Initial labeled plan/add action may create a plan; replace/remove must not silently target another date.
Several meals in same slot	Ask which entry for destructive or replacement actions; never guess.
Database/provider failure	Preserve user input; state that save/apply did not complete; avoid success confirmation.
Unknown nutrition	Leave totals unknown or mark incomplete; do not treat missing as zero.

7. Cross-feature rules

- Profile is a single current record. Historical profile versions are not currently evidenced; future analytics should not retroactively represent current profile as historical truth.
- Check-in weight and weight history are linked in current documented save behavior. A future actual meal entry must not mutate the plan or the morning check-in.
- Recipe nutrition is recipe-level. Ingredient rows only express composition; avoid summing ingredient nutrients because inventory nutrition data was not established in available docs.
- Plans and actual logs are separate entities and separate user intents. Any comparison view must label both series and exclude planned-only rows from actual intake.
- CSV imports are batch writes with preview and transaction rollback. Export semantics, filtering, file encoding and round-trip guarantees are not established; treat export as a requirement to define before implementation.
- Voice transcripts may contain sensitive health/diet information. Audio processing is through browser Web Speech; app docs say audio is not sent to Gemini/OpenAI or separately uploaded/stored, while browser vendor processing may occur.

8. Testable acceptance criteria

These criteria provide a delivery target. Criteria marked “current behavior” can be checked against the implementation; those marked “future” are proposed. No results are asserted in this document.

AC-01 Current Given valid profile fields, when saved, then one current profile is persisted and reloaded with normalized values.

AC-02 Current Given missing, extra, blank-required or out-of-range profile data, when submitted, then validation rejects it before persistence.

AC-03 Current Given a check-in date and valid weight/sleep/soreness, when saved, then that date’s check-in is retrievable and weight history reflects the saved weight.

AC-04 Current Given an invalid check-in value, when submitted, then no database write is issued.

AC-05 Current Given valid recipe CSV pair and known inventory references, when imported after confirmation, then recipes upsert and ingredient rows match the upload atomically.

AC-06 Current Given malformed CSV, duplicate recipe/food rows, invalid controlled values, or unknown references, when validated, then preview reports errors and import performs no writes.

AC-07 Current Given a meal plan date, when a meal is added/edited/deleted, then the operation is scoped to that date and order remains stable.

AC-08 Current docs; checkout verify Given reviewed text with labeled meal slots and a selected date, when applied, then the documented deterministic command handler updates that date’s plan without calling an LLM.

AC-09 Current docs; checkout verify Given ambiguous snack, repeated/missing slot, or multiple existing candidates, when command applied, then clarification is returned and no ambiguous mutation occurs.

AC-10 Future Given a planned meal only, when the user opens actual intake, then no item is pre-marked consumed; only explicit save creates an actual log.

AC-11 Future Given a correction to a logged item, when saved, then the revised value is visible and the product’s chosen audit behavior is honored.

AC-12 Future Given check-ins with missing days, when the weight chart is shown, then missing dates remain visually distinct and unit/date coverage are shown.

AC-13 Future Given reminder preferences disabled, when scheduler runs, then no reminder is sent; given enabled preferences, delivery respects timezone and schedule.

AC-14 Future Given AI suggestion generation, when context is incomplete or provider fails, then no plan/log mutation occurs and the user sees a recoverable failure.

AC-15 Future Given an AI suggestion, when displayed, then it is labeled as generated, its available context/assumptions are disclosed, and user acceptance is explicit.

AC-16 Future Given CSV export, when exported and re-imported without edits, then supported fields and recipe-food relationships round-trip without silent loss; confirm exact export contract first.

9. Open decisions and dependencies

- Confirm the exact contents and decisions in Documents 02 and 03 when available; this PDF could not read those attachments in this task context.
- Confirm whether requirements for export are CSV export of foods, recipes, meal plans, or all three; current available docs specifically establish recipe CSV import/templates, not export.
- Decide actual log quantity model, time zone/date boundary, edits/deletions and plan-linking behavior.
- Decide reminder delivery channel, runtime, opt-in language and notification schedule semantics.
- Decide charts, metrics, minimum data coverage, and nutrition provenance rules.
- Decide AI provider, privacy/consent, allowable context, safeguards, output format and user review controls.
- Check GitHub main implementation and database migrations before delivery; README/docs can describe a capability ahead of deployed Neon schema or live UI.

10. Source register and verification notes

Primary reference: GitHub repository **rd2080/Bulk_App_Personalised**, default branch **main**; README, docs/voice-input.md, docs/daily-meal-plan.md, docs/recipe-library.md, docs/user-profile-and-request-builder.md, and tests for profile, check-in, recipe library, meal plan and request builder. The project mirror supplies additional source/docs but its README and Phase 2 docs lag GitHub main on deterministic commands.

The repository files were reviewed as documentation/code evidence. Tests were not executed, the Neon database was not queried, and the deployed application was not exercised. Therefore this document distinguishes code/documented status from runtime deployment status. The PDF was generated, text-extracted for content checks, and rendered to page images for visual inspection.

End of Document 04